

Experiment 3: C Programming - Debugging Tools

1. Fundamentals of Errors and Debugging

Key Definitions

- **Error:** A mistake or blunder made by the programmer in the source code. Example: Writing `a + b` when the intention was to multiply `a * b`.
- **Bug:** A flaw or fault in a computer program that causes it to produce an incorrect or unexpected result, or to behave in unintended ways. Example: The program outputs the sum of two numbers instead of their product.
- **Debugging:** The systematic process of finding, analyzing, and removing bugs or defects in a computer program.

2. Classifications of Errors in C

Error Type	Description	Primary Cause	Example
Syntax Errors	Code violates the grammatical rules of the C language.	Missing semicolons, unmatched braces, typos in keywords.	<code>int a = 10 (missing ;)</code>
Compilation / Type Errors	Operations performed on incompatible types or missing declarations.	Assigning float string to int pointer, using undeclared variable.	<code>int *p = "hello";</code>
Linker Errors	The compiler generates object files, but the linker cannot combine them.	Function declared but not defined, missing libraries.	undefined reference to 'main'
Runtime Errors	The program crashes or terminates abnormally during execution.	Division by zero, accessing invalid memory addresses.	<code>int x = 10 / 0;</code>
Logical Errors	Code runs to completion without crashing, but output is incorrect.	Wrong mathematical formula, incorrect loop condition.	<code>for(i=0; i<5; i++)</code> instead of <code><=5</code>

Why Does a Program Compile Successfully but Produce Incorrect Output?

A compiler's job is to verify that your code obeys the syntax and type rules of the C language, and to convert that code into machine instructions. The compiler cannot understand human intent.

- If you write `sum = a - b;` instead of `sum = a + b;` the statement is completely valid C syntax. Therefore, compilation succeeds, but the resulting logic produces an erroneous output at execution time.

Case 1: Syntax & Compilation Errors

When the compiler encounters invalid code, it prints diagnostics specifying the file name, line number, column, and error description.

Example Code (sample1.c)

```
#include <stdio.h>
int main()
{
    int a = 10
    printf("%d\n", a);
    return 0;
}
```

Compilation Command & Output

```
$ cc sample1.c
sample1.c:5:5: error: expected ',' or ';' before 'printf'
```

Breakdown of Error Message

- Filename: sample1.c
- Line & Column: Line 5, Column 5
- Type of Message: error
- Description: Expected , or ; before printf statement.

```
(kali㉿kali)-[~/Desktop]
$ vi sample1.cc

(kali㉿kali)-[~/Desktop]
$ cc sample1.cc
sample1.cc: In function 'int main()':
sample1.cc:5:9: error: expected ',' or ';' before 'printf'
   5 |         printf("%d\n",a);
     |         ^~~~~~
```

```
(kali㉿kali)-[~/Desktop]
$ vi sample1.cc

(kali㉿kali)-[~/Desktop]
$ cc sample1.cc

(kali㉿kali)-[~/Desktop]
$ ./a.out
10

(kali㉿kali)-[~/Desktop]
$
```

Case 2: Compiler Warnings (-Wall, -Wextra)

A warning indicates code that is syntactically legal, but potentially dangerous or suspicious. Warnings do not stop compilation, but ignoring them frequently leads to bugs at runtime.

Example Code (sample2.c)

```
#include <stdio.h>
int main()
{
    int a;
    printf("Hello\n");
    return 0;
}
```

Standard Compilation vs. Warning Flags

```
# Standard compilation (may suppress minor warnings)
$ cc sample2.c
$ ./a.out
Hello

# Enabling comprehensive warning flags
$ cc -Wall -Wextra sample2.c
sample2.c:5:9: warning: unused variable 'a' [-Wunused-variable]
```

- -Wall: Enables a broad set of commonly useful compiler warnings.
- -Wextra: Enables additional warnings not covered by -Wall.

```
(kali㉿kali)-[~/Desktop]
$ vi sample2.cc

(kali㉿kali)-[~/Desktop]
$ cc sample2.cc

(kali㉿kali)-[~/Desktop]
$ ./a.out
Hello
```

```
(kali㉿kali)-[~/Desktop]
$ cc -Wall -Wextra sample2.cc
sample2.cc: In function 'int main()':
sample2.cc:4:13: warning: unused variable 'a' [-Wunused-variable]
   4 |         int a;
     |             ^
```

Case 3: printf-Based Debugging (Logical bug)

Before using complex debugging tools, simple program flow and state inspection can be performed using output statements (printf).

Basic Value Tracing

```
printf("DEBUG: a = %d, b = %d\n", a, b);
```

Example: Tracking Loop Execution (sample3.c)

The following program adds numbers from 1 to 4 instead of 1 to 5 due to a logical bug. Debug printf() statements help expose the exact iteration behavior.

```

#include <stdio.h>
int main(void)
{
    int i, sum = 0;
    for(i = 0; i < 5; i++)
    {
        printf("DEBUG: entering iteration i = %d\n", i);
        sum = sum + i;
        printf("DEBUG: sum updated to = %d\n", sum);
    }
    printf("Final Sum = %d\n", sum);
    return 0;
}

```

3. Introduction to GDB (GNU Debugger)

Background: What is GNU & GDB?

- GNU: A recursive acronym for "GNU's Not Unix!". It is an open-source operating system project initiated by Richard Stallman in 1983 to provide a completely free Unix-like system.
- GDB (GNU Debugger): The standard debugger for the GNU software system, allowing developers to inspect variable values, set execution breakpoints, and analyze crashes.

Installation & Compilation for GDB

```

# Update package list and install GDB on Linux systems
$ sudo apt update
$ sudo apt install gdb

# Compile source code with debugging symbols enabled (-g)
$ cc -g -Wall -Wextra program.c -o program

# Launch GDB with the compiled binary
$ gdb ./program

```

```
(kali@kali)-[~/Desktop]
$ cc -g -Wall -Wextra sample3.cc -o sample3

(kali@kali)-[~/Desktop]
$ gdb ./sample3
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./sample3...
(gdb)
```

```
(kali@kali)-[~/Desktop]
$ cc -g -Wall -Wextra sample3.cc -o sample3

(kali@kali)-[~/Desktop]
$ gdb ./sample3
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
```

```
(kali@kali)-[~/Desktop]
$ cc -g -Wall -Wextra sample3.cc -o sample3

(kali@kali)-[~/Desktop]
$ gdb ./sample3
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
```

```
(kali@kali)-[~/Desktop]
$ cc -g -Wall -Wextra sample3.cc -o sample3

(kali@kali)-[~/Desktop]
$ gdb ./sample3
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./sample3...
(gdb)
```

Quick Reference: Common GDB Commands

Command	Shorthand	Description / Purpose
break <line / func>	b	Sets a breakpoint at a specified line number or function name.
run	r	Starts execution of the program from the beginning.
next	n	Executes the next line of code (steps over function calls).
step	s	Executes the next line of code (steps into function calls).
print <var>	p	Prints the current value of a variable or expression.
display <var>	disp	Automatically prints the value of a variable after every step.
continue	c	Resumes execution until the next breakpoint or program completion.

list	l	Prints source code lines around the current execution position.
info locals	info loc	Lists all local variables and their current values in the current frame.
backtrace	bt	Displays the current call stack (chain of function calls).
info breakpoints	i b	Displays all active breakpoints and their numbers.
delete <id>	d	Deletes the specified breakpoint number (or all if omitted).
disable <id>	dis	Temporarily disables a breakpoint without deleting it.
enable <id>	en	Re-enables a previously disabled breakpoint.
quit	q	Exits the GDB debugging session.

Case 4: Debugging a Logical Error using GDB

Example: Code with Logic Bug (sample4.c)

```

#include <stdio.h>           // Line 1
int main()                  // Line 2
{
    // Line 3
    int i, sum = 0;          // Line 4
    for(i = 1; i < 5; i++)    // Line 5 (Logical Bug: should be i <= 5)
    {
        // Line 6
        sum = sum + i;       // Line 7
    }
    // Line 8
    printf("Sum = %d\n", sum); // Line 9
    return 0;                // Line 10
}

```

- Expected Output: 15 (1 + 2 + 3 + 4 + 5)
- Actual Output: 10 (1 + 2 + 3 + 4)

Debugging Walkthrough in GDB

```

$ cc -g sample4.c -o sample4
$ gdb ./sample4
(gdb) list
(gdb) break 7

```

Breakpoint 1 at 0x1149: file sample4.c, line 7.

```
(gdb) run
Starting program: ./sample4
Breakpoint 1, main () at sample4.c:7
7  sum = sum + i;

(gdb) print i
$1 = 1
(gdb) display i
1: i = 1
(gdb) display sum
2: sum = 0
(gdb) continue
...
1: i = 4
2: sum = 6
(gdb) continue
Sum = 10
[Inferior 1 (process) exited normally]
```

Conclusion: The loop terminates when *i* reaches 5 without executing the loop body for 5. The loop condition should be changed from *i* < 5 to *i* <= 5.

Case 5: Debugging Functions (next vs. step)

- next: Executes the line. If the line contains a function call, the function executes entirely in the background without entering its body.
- step: Steps into the function call, transferring GDB's focus into the function body for line-by-line inspection.

Example Code (sample5.c)

```
#include <stdio.h>
int square(int x)
{
    int result1;
    result1 = x * x;
    return result1;
}
int main()
{
    int result, n = 5;
    result = square(n);
    printf("Square = %d\n", result);
    return 0;
}
```

```
}
```

Option A: Stepping Over with next

```
(gdb) break main
(gdb) run
(gdb) next # Moves to result = square(n);
(gdb) next # Executes square(n) completely
(gdb) print result
$1 = 25 # Result obtained without stepping inside square()
```

Option B: Stepping Into with step

```
(gdb) break main
(gdb) run
(gdb) next # Moves to result = square(n);
(gdb) step # Enters square(x) function body!
square (x=5) at sample5.c:5
5  result1 = x * x;
(gdb) print x
$1 = 5
(gdb) next
(gdb) print result1
$2 = 25
```

```

(kali㉿kali)-[~/Desktop]
$ vi sample5.cc

(kali㉿kali)-[~/Desktop]
$ cc -g sample5.cc -o sample5

(kali㉿kali)-[~/Desktop]
$ gdb ./sample5
GNU gdb (Debian 17.2-1+b1) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word" ...
Reading symbols from ./sample5...
(gdb) break main
Breakpoint 1 at 0x1156: file sample5.cc, line 10.
(gdb) run
Starting program: /home/kali/Desktop/sample5

```

```

(gdb) run
Starting program: /home/kali/Desktop/sample5
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, main () at sample5.cc:10
10      int result, n = 5;
(gdb) next
11      result = square(n);
(gdb) next
12      printf("Square = %d\n", result);
(gdb) print result
$1 = 25
(gdb)

```

Case 6: Debugging Runtime Errors using Backtrace (backtrace / bt)

When a program crashes (e.g., floating point exception / division by zero), GDB can pinpoint the exact function call sequence leading to the crash.

Example Code (sample6.c)

```
#include <stdio.h>
int divide(int a, int b)
{
    return a / b;    // Division by zero when b = 0
}
int calculate(int x)
{
    return divide(x, 0); // Passes 0 as divisor
}
int main()
{
    int result = calculate(10);
    printf("Result: %d\n", result);
    return 0;
}
```

GDB Backtrace Execution

```
$ cc -g sample6.c -o sample6
$ gdb ./sample6
(gdb) run
Program received signal SIGFPE, Arithmetic exception.
0x0000555555555132 in divide (a=10, b=0) at sample6.c:5
5   return a / b;
(gdb) backtrace
#0 ... in divide (a=10, b=0) at sample6.c:5
#1 ... in calculate (x=10) at sample6.c:10
#2 ... in main () at sample6.c:15
```

Understanding Stack Frames

- Frame #0: The function where the crash actually occurred (divide).
- Frame #1: The function that called frame #0 (calculate).
- Frame #2: The top-level function that started the call sequence (main).

```
(gdb) frame 0
(gdb) print b    # Shows b = 0, identifying division by zero
(gdb) frame 1
(gdb) print x    # Shows x = 10
```

Case 7: Debugging Array Out-of-Bounds Errors using GDB

In C, array bounds are not checked at runtime automatically. Accessing indices beyond the declared limit results in undefined behavior or memory corruption.

Example Code (sample7.c)

```
#include <stdio.h>
int main()
{
    int a[5];
    int i;
    for(i = 0; i <= 5; i++)
    {
        scanf("%d", &a[i]); // Line 8
    }
    return 0;
}
```

Inspection in GDB

```
(gdb) break 8
(gdb) run
(gdb) print &a[0]
$1 = (int *) 0x7fffffff9d9c0
(gdb) print &a[4]
$2 = (int *) 0x7fffffff9d9d0
(gdb) print &a[5]
$3 = (int *) 0x7fffffff9d9d4 # Valid indices are 0..4; index 5 is out of bounds!
```

- Correct Loop Condition: `for(i = 0; i < 5; i++)`

Case 8: Segmentation Faults (SIGSEGV) using GDB

A Segmentation Fault occurs when a program attempts to access a memory location that it does not have permission to access (such as dereferencing a NULL pointer).

Example Code (sample8.c)

```
#include <stdio.h>
int main()
{
    int *p = NULL; // Pointer points to address 0x0
    *p = 10;      // Attempting to write to address 0x0 causes Segmentation Fault
    printf("%d\n", *p);
    return 0;
}
```

Debugging in GDB

```
$ cc -g sample8.c -o sample8
$ gdb ./sample8
(gdb) run
Program received signal SIGSEGV, Segmentation fault.
0x0000555555555139 in main () at sample8.c:6
6  *p = 10;
(gdb) print p
$1 = (int *) 0x0
(gdb) print *p
Cannot access memory at address 0x0
```

Corrected Code

```
#include <stdio.h>
int main()
{
    int x;
    int *p = &x; // Assign p to point to valid memory address of x
    *p = 10;    // Valid write
    printf("%d\n", *p);
    return 0;
}
```

4. Systematic Debugging Strategy

To debug effectively, follow this 8-step methodology:

1. Observe Expected vs. Actual Output
2. Reproduce the Issue Consistently
3. Compile with Flags (-Wall -Wextra -g)
4. Isolate Suspicious Code Sections
5. Inspect State (printf or GDB breakpoints)
6. Apply a Single Minimal Fix
7. Recompile and Re-test
8. Verify with Edge Cases & Boundary Inputs